Sensory Weighting System Technical Design

Comfort Curve + Genome Weighting Implementation Specification

Zixun Zhou · RCA Graduate Diploma, Unit 3 Investigate / Unit 4 Realise

0. Where This Module Sits in the Overall Architecture
1. Layer One: The Comfort Curve
2. Layer Two: Genome Weights
3. Genome Configuration File Specification (JSON Schema)
4. Auto-Calibration of Comfort Bands (replacing hand-written estimates)
5. Full Data Pipeline (pseudocode)
6. Edge Case Handling
7. Explicitly Excluded from the Five-Week Scope (maintaining honest boundaries)
8. Suggested File Structure

0. Where This Module Sits in the Overall Architecture

```
Raw sensor readings (7 channels)
        |
        v
[Module defined in this document]
  Layer 1: per-channel comfort curve  -> converts a physical value into
           "degree of deviation from comfort" (deviation)
  Layer 2: per-channel weight          -> determines how much this
           deviation matters to the current genome
        |
        v
Weighted deviation vector (7-dim) + remaining parameters
from the existing 8-parameter genome system
        |
        v
CfC + AutoNCP core (unmodified — simply consumes this vector)
```

Key principle: this layer is a pure Python pre-processing step. It involves no training and does not touch the internals of `ncps`. The CfC core itself has no notion that "genomes" exist at all — it is simply processing an input vector that happens to change whenever the genome is switched. This is the path we confirmed is achievable within a five-week timeline without retraining anything.

---

1. Layer One: The Comfort Curve

1.1 Why a weight alone is not enough — a curve is required

A single scalar weight can only express "how much this channel matters" — it cannot express the non-linear relationship of "what range of values feels comfortable, and how severe the stress becomes beyond it." A tomato plant is comfortable at pH 6.4; pH 6.0 and pH 6.8 are still tolerable; pH 5.0 is severe stress. That is a curve, not a number.

1.2 Curve shape: trapezoidal membership function

Each channel is defined by four parameters, ordered low to high: `stress_lo, lo, hi, stress_hi`.

```
comfort(v)
  1.0 |        .--------.
      |       /          \
      |      /            \
  0.0 +-----.--------------.-----> v
    stress_lo  lo        hi   stress_hi
```

- Within `[lo, hi]`: fully comfortable, `comfort = 1.0`
- Between `lo` and `stress_lo` (or `hi` and `stress_hi`): linear decrease
- Beyond `stress_lo` / `stress_hi`: treated as full stress, `comfort = 0.0`, does not continue to worsen (prevents an extreme reading from blowing out the signal)

1.3 Signed deviation (preserving the direction of "too much" vs. "too little")

Outputting only a `comfort` score loses directional information — moisture that is too high and moisture that is too low both give `comfort = 0`, but for a tomato plant these are two entirely different stressors (waterlogging vs. drought). The network should be able to distinguish them. So what is actually fed to the network is a **signed deviation**:

```
deviation(v) =
    (v - lo) / (lo - stress_lo)   if v < lo   -> negative, more negative = more severe
    0                              if lo <= v <= hi
    (v - hi) / (stress_hi - hi)   if v > hi   -> positive, more positive = more severe
```

Output is clipped to the range `[-1, 1]`.

1.4 Python implementation

```
        def comfort_deviation(value, lo, hi, stress_lo, stress_hi):
            """
            Returns a signed deviation, range [-1, 1].
            0 = fully comfortable; negative = below the comfort band
            (more negative = more severe); positive = above the comfort band.
            """
            if lo <= value <= hi:
                return 0.0
            elif value < lo:
                if value <= stress_lo:
                    return -1.0
                return -(lo - value) / (lo - stress_lo)
            else:  # value > hi
                if value >= stress_hi:
                    return 1.0
                return (value - hi) / (stress_hi - hi)
```

---

2. Layer Two: Genome Weights

2.1 Relationship to the existing 8-parameter system

In the existing genome system, the "sensory threshold multiplier" is currently **a single global scalar** — one number controlling overall sensory sensitivity. This change upgrades it **from a scalar to a 7-dimensional vector**, with one weight per sensor channel. The remaining seven parameters (arousal baseline, emotional inertia, approach bias, etc.) are **left entirely unchanged and reused directly**.

In other words, this is not a new system — it is taking one existing parameter and expanding it from "one number" to "seven numbers." Architecturally, it is a natural extension of the existing system, not a rebuild.

2.2 What the weight does

```
        weighted_signal = [w[i] * deviation[i] for i in range(7)]
```

A channel with weight 0 contributes 0 to the weighted output regardless of how large its deviation is — the network "cannot feel" stress along that dimension, even though the raw data continues to be collected and logged (used for the auto-calibration in Section 4, and for the future environment-type detection feature).

---

3. Genome Configuration File Specification (JSON Schema)

Each genome is an independent JSON file, containing the channel configuration plus the remaining parameters from the existing 8-parameter system.

3.1 Full field specification

```
        {
          "genome_id": "tomato_soil",
          "display_name": "Tomato / Soil Proxy",

          "channels": {
            "temperature":   {"active": true,  "weight": 0.8, "lo": 18, "hi": 28, "stress_lo": 5,   "stress_hi": 40},
            "moisture":      {"active": true,  "weight": 0.9, "lo": 40, "hi": 70, "stress_lo": 10,  "stress_hi": 95},
            "voc":           {"active": true,  "weight": 0.6, "lo": 0,  "hi": 50, "stress_lo": 0,   "stress_hi": 200},
            "humidity_air":  {"active": true,  "weight": 0.3, "lo": 40, "hi": 70, "stress_lo": 10,  "stress_hi": 95},
            "light":         {"active": true,  "weight": 0.7, "lo": 8000, "hi": 30000, "stress_lo": 500, "stress_hi":
60000},
            "sound":         {"active": true,  "weight": 0.15,"lo": 0,  "hi": 40, "stress_lo": 0,   "stress_hi": 90},
            "ph":            {"active": true,  "weight": 0.7, "lo": 6.0,"hi": 6.8,"stress_lo": 4.5, "stress_hi": 8.5},
            "ec":            {"active": true,  "weight": 0.6, "lo": 800,"hi": 1500,"stress_lo": 200,"stress_hi": 3000}
          },

          "baseline_params": {
            "arousal_baseline": 0.3,
            "emotional_inertia": 0.6,
            "approach_bias": 0.5,
            "_comment": "The remaining five parameters follow the definitions and values already established in the
existing 8-parameter genome system"
          }
        }
```

Note: the `lo/hi/stress_lo/stress_hi` values shown above are the estimates carried over from the previous design document — **these are design assumptions, not verified scientific thresholds.** Section 4 provides an honest, implementable calibration method to replace "estimates I made up."

3.2 The role of the `active` field

For a channel with `active: false`, the Layer 1 comfort curve **is not computed at all** — it is not that the weight is zero, it is that the entire computation is skipped. This corresponds to the `moisture` channel in the freshwater genome — the probe is physically not in a soil substrate, so not only should the weight be zero, but computing "how far it deviates from a comfort band" is meaningless in the first place. Distinguishing `active = false` from `weight = 0` keeps the code logic honest: the former means "this channel has no physical meaning for this ecological niche," the latter means "this channel is meaningful, but the current genome does not care about it."

---

4. Auto-Calibration of Comfort Bands (replacing hand-written estimates)

This section directly addresses the requirement to flag the earlier estimates as "unverified" — not merely with a disclaimer, but with an executable calibration procedure, so that numbers produced within the five-week timeline have a real basis rather than being something I invented.

4.1 Procedure

Once the device is deployed to a real environment, **the first 3–7 days involve no stress judgements at all — only data logging** (during this period the expression layer can hold a neutral state, e.g. the LED ring maintaining a soft white glow, signalling "still observing, not yet forming a judgement").

```
        import numpy as np

        def calibrate_comfort_band(readings, margin_std=1.0, stress_std=3.0):
            """
            readings: raw readings for a given channel over the past N days
            Returns the calibrated (lo, hi, stress_lo, stress_hi) for that channel
            """
            mean = np.mean(readings)
            std = np.std(readings)
            lo = mean - margin_std * std
            hi = mean + margin_std * std
            stress_lo = mean - stress_std * std
            stress_hi = mean + stress_std * std
            return lo, hi, stress_lo, stress_hi
```

The mean and standard deviation from this observation window are used to automatically generate `lo/hi/stress_lo/stress_hi`, rather than applying the literature-based estimates given earlier. Those literature estimates can serve as a **fallback default before calibration** (in case something goes wrong with the environment during the calibration window itself, at least there is a roughly reasonable range) — but the thresholds actually used for demo judgements should come from what the device itself observed in that specific environment.

4.2 The added value of this design

This procedure incidentally resolves two issues:

First, an honesty issue — if a reviewer asks "where did these thresholds come from," the accurate answer is "the device auto-calibrated them by observing the real data from the deployment environment over its first few days — they are not hard-coded generic thresholds." This holds up far better than "I estimated them from some references."

Second, a narrative benefit — "observe and adapt for a few days first, then form a judgement of what 'normal' looks like for this environment" is itself a very natural developmental-stage narrative, and maps directly onto the earliest stage of the existing virtual-age/developmental system. This is not an added feature — it is the "newborn first becomes familiar with its environment" narrative, already present in the project, turned into a genuinely useful calibration mechanism.

---

5. Full Data Pipeline (pseudocode)

```python
import json

class SensoryMapper:
    def __init__(self, genome_path):
        with open(genome_path) as f:
            self.genome = json.load(f)

    def process(self, raw_readings: dict) -> list:
        """
        raw_readings: {"temperature": 22.3, "moisture": 55.0, ...}
        Returns the weighted deviation vector fed to the CfC core,
        in a fixed channel order
        """
        channel_order = ["temperature", "moisture", "voc", "humidity_air",
                         "light", "sound", "ph", "ec"]
        output = []
        for ch in channel_order:
            cfg = self.genome["channels"][ch]
            if not cfg["active"]:
                output.append(0.0)
                continue
            raw_value = raw_readings.get(ch)
            if raw_value is None:
                # Sensor fault / missing reading: log a warning,
                # feed 0 (neutral) for this tick
                output.append(0.0)
                continue
            dev = comfort_deviation(raw_value, cfg["lo"], cfg["hi"],
                                    cfg["stress_lo"], cfg["stress_hi"])
            output.append(cfg["weight"] * dev)
        return output
```

The integration point is straightforward: in the existing `life_v2.py` main loop, wherever sensor/environment readings were previously fed directly to the CfC core, this step now inserts `SensoryMapper.process()` in between. The output dimensionality is unchanged (still one value per channel), so the CfC side of the code requires no changes at all.

---

6. Edge Case Handling

Case	Handling
Missing / faulty sensor reading	Log a warning; the channel is fed 0 (neutral, no stress judged) for that tick
Reading outside physically possible range (e.g. pH reads 15)	Treated as a faulty reading, handled the same as above; not passed into the curve calculation
Channel with `active = false`	Curve calculation is skipped, fixed output of 0; raw reading is still written to the log
Still within the calibration period (fewer than 7 days of data)	Fall back to the default estimated values in the JSON, while flagging that calibration is still in progress

---

7. Explicitly Excluded from the Five-Week Scope (maintaining honest boundaries)

- **Per-sensor rate-of-change (trend/derivative) features**: an interface is reserved architecturally (`SensoryMapper` could be extended to output both deviation and rate of change), but this is not a required part of the five-week MVP. The current design uses only instantaneous readings — the principle of "rate of change over absolute value" is currently only embodied in the calibration logic, not in real-time inference. If time allows during the demo, a simple sliding-window difference could be added as an eighth input dimension.
- **Automatic genome-to-environment matching detection**: out of scope for this implementation. The calibration data from Section 4 could support this feature in the future, but it will not be built within the five weeks.

---

8. Suggested File Structure

```
project/
|-- genomes/
|   |-- tomato_soil.json
|   |-- freshwater.json
|   `-- indoor_air.json
|-- sensory_mapper.py      # the two-layer transform defined in this document
|-- calibration.py         # the auto-calibration script from Section 4
|-- life_v2.py              # existing main loop, with SensoryMapper call inserted
`-- logs/
    `-- raw_sensor_log.jsonl   # full raw data across all channels, including zero-weight channels
```

The benefit of this split is that genome configuration and the core inference code are fully decoupled: switching genomes means swapping a JSON file path, with no changes required to any Python logic — consistent with the “chameleon” design’s claim of “a single brain, pluggable sensory perspectives.”